

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_736be611-fb5\agent-ad58e512c153e84da.jsonl`  
- SHA-256: `99b7e355da6bf46840ec0760620d22fd8f8c161eaab5f319ee4e0934a04608d4`  
- Source modified: `2026-06-16T15:16:55+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_736be611-fb5`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T15:14:30.300Z)

Adversarially VERIFY this finding from the real-footage rally-detection validation. Try hard to REFUTE or show it is overstated, using ONLY the data (you MAY Read output/local_vs_gemini_6.json). Default to holds=false if the numbers do not clearly support it.

CLAIM (trajectory-signal): GX010128 over-segments ~2x because its true rallies are short and dense (Gemini mean 5.85s, 39 rallies) yet within each rally the shuttle briefly stalls/occludes, so the 12s cap finds a sub-rally inactivity gap and chops one true rally into two-plus windows. EVIDENCE CITED: GX010128 W1 n=78 vs Gemini 39 (count_ratio 2.0); matched=35 but splits=4 and local_only=34 with mean window 7.8s ? windows SHORTER than Gemini's 5.85s mean would predict for clean 1:1, and per-video mean 7.8s is the lowest of the six, indicating the cap+gap split is firing inside rallies. mIoU only 0.53 despite 35 matches.

REAL-FOOTAGE VALIDATION DATA (6 Kushagra/Yash June-12 clips, 1080p proxies, fully-LOCAL no-AI WASB windowing vs Gemini. Gemini = strong REFERENCE, NOT ground truth ? these 6 have no golden labels EXCEPT mahadevpura-2 which the owner hand-labeled, see GROUND-TRUTH below).

Rally COUNT (Gemini | local baseline | local W1 cap=12 | local W1+W2 mc=1):

```
GX010128: 39 | 15 | 78 | 75  
GX010129: 27 | 2 | 50 | 47  
GX020125: 11 | 6 | 18 | 15  
mahadevpura-0: 40 | 19 | 62 | 43  
mahadevpura-1: 9 | 1 | 2 | 2  
mahadevpura-2: 39 | 5 | 40 | 40  
AGGREGATE: Gemini 165 | baseline 48 (mean window 62.9s, 27 merge-groups, mIoU 0.23) |  
            W1 250 (mean 10.8s, 10 merge-groups, mIoU 0.39) |  
            W1+W2 222 (mean 11.5s, 10 merge-groups, mIoU 0.39, 6 gemini-only "misses")  
Per-video mean window (W1): GX010128 7.8s, GX010129 9.3s, GX020125 14.1s, mahadevpura-0 10.4s,  
mahadevpura-1 86.9s (the continuously-active blob W1 cannot cap ? no inactivity gap),  
mahadevpura-2 13.8s. Gemini means ~5.7-7.2s.
```

GROUND-TRUTH (mahadevpura-2, 40 owner-labeled rallies, mean 5.7s, verified frame-aligned):

```
F1@0.5 / F1@0.25 / mean-IoU: Gemini 0.557/0.835/0.55 ; local W1+W2 0.150/0.500/0.31 ;  
local W1+W2+hard-cap 0.296/0.722/0.50 (best local). baseline 0.000/0.000/0.03.  
BOUNDARY ERROR (matched rallies, seconds): Gemini |?start| med 1.46 |?end| med 1.19 ;  
local-hardcap |?start| med 1.74 (~= Gemini!) |?end| med 3.12 (>> Gemini).  
=> local's rally-START detection is ~at par with Gemini; the gap is the rally-END.  
IoU<->time: for a 5.7s rally, tIoU 0.5 tolerates only ~+-1.9s shift (the ~1-1.5s service window  
nearly exhausts it -> tIoU 0.5 over-penalizes; owner wants a +-2s tolerance metric).
```

PRIOR GOLDEN DELTAS (n=6, then n=7 incl. mahadevpura-2): baseline F1 0.300 -> W1 0.439 (+0.139,

6/0)
-> W1+W2 0.457 (+0.019, 6/0); hard-cap golden-neutral on n=6 (+0.006 n.s.) but became BEST on n=7
(continuously-active clips). All flags default-OFF, promoted only on measured LOVO lift + sign-test.

WINDOWING KNOBS: a frame is "active" iff shuttle visible AND per-second-normalized velocity > velocity_thresh(0.02); active frames within merge_gap(2.0s) coalesce; min_window(2.0s). W1 = max_window_duration cap(12s) split at the largest internal inactivity gap. W1.5 hard-cap = midpoint-chop when no gap. W2 = net-crossings gate (drop windows with <1 net crossing).

2. assistant (2026-06-16T15:14:32.692Z)

I'll verify this claim by examining the actual data file.

3. user (2026-06-16T15:14:33.654Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\w2-safeguard.

4. assistant (2026-06-16T15:14:36.352Z)

The file isn't at the expected path. Let me locate it.

5. user (2026-06-16T15:14:36.880Z)

No files found

6. user (2026-06-16T15:14:39.636Z)

eval_baselines\gemini_wrapper_2026-06-10.json

7. assistant (2026-06-16T15:14:43.327Z)

The cited file doesn't exist in this worktree. Let me search more broadly for the data, including artifacts and output directories.

8. user (2026-06-16T15:14:43.404Z)

No files found

9. user (2026-06-16T15:14:47.373Z)

Exit code 2
total 292
drwxr-xr-x 1 avidu 197609 0 Jun 16 20:34 .
drwxr-xr-x 1 avidu 197609 0 Jun 16 11:24 ..
drwxr-xr-x 1 avidu 197609 0 Jun 16 11:14 chunks
drwxr-xr-x 1 avidu 197609 0 Jun 16 20:38 highlights_work
-rw-r--r-- 1 avidu 197609 212992 Jun 16 20:34 val.db
-rw-r--r-- 1 avidu 197609 61195 Jun 16 20:38 val_extract.log
drwxr-xr-x 1 avidu 197609 0 Jun 16 20:38 val_reels
drwxr-xr-x 1 avidu 197609 0 Jun 16 20:34 wasb
---ARTIFACTS---

10. assistant (2026-06-16T15:14:50.759Z)

The `output/local\_vs\_gemini\_6.json` file does not exist. Let me check the artifacts directory (referenced in git status) and search the whole tree for any file containing the per-video data.

11. user (2026-06-16T15:14:56.243Z)

===SEARCH GX010128===

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/highlights_work/GX010128 - run.log
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/highlights_work/_batch_run.log
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/val.db
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/val_extract.log
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/val_reels/GX010128 - run.log
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/val_reels/_batch_run.log
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard/output/wasb/GX010128__db84f3e65318_wasb_wasbcache/manifest.json
```

12. assistant (2026-06-16T15:15:00.220Z)

The cited file `output/local\_vs\_gemini\_6.json` does not exist anywhere in the worktree. The only structured per-video data I can find is `output/val.db` (a SQLite DB) and various logs. The validation results JSON that the claim cites as its evidence source is simply not present. Let me check whether the data can be reconstructed from val.db or the logs, and search the broader repo (parent, not just worktree).

13. user (2026-06-16T15:15:54.047Z)

```
=== count_ratio / splits / local_only / mIoU search across whole repo ===
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/.mypy_cache/3.13/cache.7.db
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/.pytest_cache/v/cache/nodeids
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/ablation.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/batch.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/eval_partial_labels.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/experiment.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/harness.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/segmentation_metrics.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/served_gate_a.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/ablation.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/batch.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/eval_partial_labels.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/experiment.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/harness.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/segmentation_metrics.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/served_gate_a.cpython-313.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-first-ab-experiment/README.md
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-human-gt-tuning/README.md
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/EVALUATION.md
```

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/GOLDEN_SET_VENDORING.md
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/QUALITY_ITERATIONS.md
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/eval_baselines/experiment_leaderboard.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_eval_batch.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_experiment_honest.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_segmentation_metrics.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_served_gate_a_regression.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/tests/__pycache__/test_eval_batch.cpython-313-pytest-9.0.3.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/tests/__pycache__/test_experiment_honest.cpython-313-pytest-9.0.3.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/tests/__pycache__/test_segmentation_metrics.cpython-313-pytest-9.0.3.pyc
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/tests/__pycache__/test_served_gate_a_regression.cpython-313-pytest-9.0.3.pyc
=== any local_vs_gemini anywhere on parent repo ===
C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.md
C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_report.md
C:/Users/avidu/Projects/badminton-highlight-indexer/scratch/compare_local_vs_gemini.py

```

14. assistant (2026-06-16T15:15:57.918Z)

The cited file exists in the parent repo at `C:/Users/avidu/Projects/badminton-highlight-indexer/output/local\_vs\_gemini\_6.json` (not in this worktree's `output/`). Let me read it.

15. user (2026-06-16T15:15:58.469Z)

```

1      [
2      {
3          "name": "GX010128",
4          "vid": "db84f3e65318",
5          "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010128__db84f3e65318_wasb.csv",
6          "n_gem": 39,
7          "play_gem": 228.0,
8          "mean_gem": 5.846153846153846,
9          "variants": {
10             "baseline": {
11                 "n": 15,
12                 "play": 711.4614614614616,
13                 "mean_dur": 47.430764097430774,
14                 "max_dur": 133.2499165832499,
15                 "count_ratio": 0.38461538461538464,
16                 "matched": 4,
17                 "local_only": 1,
18                 "gemini_only": 0,
19                 "splits": 0,
20                 "merges": 10,
21                 "mean_iou": 0.3564105364413525,
22                 "gem_rallies_in_merges": 35
23             },
24             "w1": {
25                 "n": 78,
26                 "play": 605.1051051051056,
27                 "mean_dur": 7.757757757757765,
28                 "max_dur": 16.616616616616625,
29                 "count_ratio": 2.0,

```

```

30         "matched": 35,
31         "local_only": 34,
32         "gemini_only": 0,
33         "splits": 4,
34         "merges": 0,
35         "mean_iou": 0.5294639963484731,
36         "gem_rallies_in_merges": 0
37     },
38     "w1+w2": {
39         "n": 75,
40         "play": 594.494494494495,
41         "mean_dur": 7.926593259926601,
42         "max_dur": 16.616616616616625,
43         "count_ratio": 1.9230769230769231,
44         "matched": 35,
45         "local_only": 31,
46         "gemini_only": 0,
47         "splits": 4,
48         "merges": 0,
49         "mean_iou": 0.5294639963484731,
50         "gem_rallies_in_merges": 0
51     }
52 },
53 "status": "ok"
54 },
55 {
56     "name": "GX010129",
57     "vid": "99a3e73afde3",
58     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010129__99a3e73afde3_wasb.csv",
59     "n_gem": 27,
60     "play_gem": 195.6,
61     "mean_gem": 7.244444444444445,
62     "variants": {
63         "baseline": {
64             "n": 2,
65             "play": 534.4177510844178,
66             "mean_dur": 267.2088755422089,
67             "max_dur": 522.6893560226894,
68             "count_ratio": 0.07407407407407407,
69             "matched": 0,
70             "local_only": 1,
71             "gemini_only": 0,
72             "splits": 0,
73             "merges": 1,
74             "mean_iou": 0.0,
75             "gem_rallies_in_merges": 27
76         },
77         "w1": {
78             "n": 50,
79             "play": 463.7137137137137,
80             "mean_dur": 9.274274274274273,
81             "max_dur": 27.84451117784451,
82             "count_ratio": 1.8518518518518519,
83             "matched": 23,
84             "local_only": 18,
85             "gemini_only": 0,
86             "splits": 4,
87             "merges": 0,
88             "mean_iou": 0.4819655005521238,
89             "gem_rallies_in_merges": 0
90         },
91         "w1+w2": {
92             "n": 47,
93             "play": 447.0303636970303,

```

```

94         "mean_dur": 9.511284333979368,
95         "max_dur": 27.84451117784451,
96         "count_ratio": 1.7407407407407407,
97         "matched": 22,
98         "local_only": 16,
99         "gemini_only": 1,
100        "splits": 4,
101        "merges": 0,
102        "mean_iou": 0.48514996335815863,
103        "gem_rallies_in_merges": 0
104    }
105 },
106 "status": "ok"
107 },
108 {
109     "name": "GX020125",
110     "vid": "407fa7e7e9f1",
111     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\GX020125__407fa7e7e9f1_wasb.csv",
112     "n_gem": 11,
113     "play_gem": 71.0,
114     "mean_gem": 6.454545454545454,
115     "variants": {
116         "baseline": {
117             "n": 6,
118             "play": 280.6806806806807,
119             "mean_dur": 46.78011344678012,
120             "max_dur": 82.66599933266599,
121             "count_ratio": 0.5454545454545454,
122             "matched": 1,
123             "local_only": 2,
124             "gemini_only": 0,
125             "splits": 0,
126             "merges": 3,
127             "mean_iou": 0.40270862800565765,
128             "gem_rallies_in_merges": 10
129         },
130         "w1": {
131             "n": 18,
132             "play": 254.03737070403744,
133             "mean_dur": 14.113187261335414,
134             "max_dur": 40.05672339005673,
135             "count_ratio": 1.6363636363636365,
136             "matched": 9,
137             "local_only": 8,
138             "gemini_only": 0,
139             "splits": 0,
140             "merges": 1,
141             "mean_iou": 0.4329964346464835,
142             "gem_rallies_in_merges": 2
143         },
144         "w1+w2": {
145             "n": 15,
146             "play": 246.5131798465132,
147             "mean_dur": 16.434211989767547,
148             "max_dur": 40.05672339005673,
149             "count_ratio": 1.3636363636363635,
150             "matched": 9,
151             "local_only": 5,
152             "gemini_only": 0,
153             "splits": 0,
154             "merges": 1,
155             "mean_iou": 0.4329964346464835,
156             "gem_rallies_in_merges": 2
157         }

```

```

158     },
159     "status": "ok"
160 },
161 {
162     "name": "mahadevpura-0",
163     "vid": "6460c2fc7647",
164     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\mahadevpura-0__6460c2fc7647_wasb.csv",
165     "n_gem": 40,
166     "play_gem": 262.0,
167     "mean_gem": 6.55,
168     "variants": {
169         "baseline": {
170             "n": 19,
171             "play": 711.5615615615616,
172             "mean_dur": 37.45060850324008,
173             "max_dur": 134.13413413413414,
174             "count_ratio": 0.475,
175             "matched": 6,
176             "local_only": 4,
177             "gemini_only": 0,
178             "splits": 0,
179             "merges": 9,
180             "mean_iou": 0.3446682872977475,
181             "gem_rallies_in_merges": 34
182         },
183         "W1": {
184             "n": 62,
185             "play": 646.1127794461132,
186             "mean_dur": 10.421173862034085,
187             "max_dur": 47.66433099766434,
188             "count_ratio": 1.55,
189             "matched": 34,
190             "local_only": 22,
191             "gemini_only": 0,
192             "splits": 2,
193             "merges": 2,
194             "mean_iou": 0.4928109196809172,
195             "gem_rallies_in_merges": 4
196         },
197         "W1+W2": {
198             "n": 43,
199             "play": 549.5161828495164,
200             "mean_dur": 12.779446112779452,
201             "max_dur": 47.66433099766434,
202             "count_ratio": 1.075,
203             "matched": 30,
204             "local_only": 9,
205             "gemini_only": 5,
206             "splits": 1,
207             "merges": 2,
208             "mean_iou": 0.4837616679798798,
209             "gem_rallies_in_merges": 4
210         }
211     },
212     "status": "ok"
213 },
214 {
215     "name": "mahadevpura-1",
216     "vid": "38bef75d78cb",
217     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-1__38bef75d78cb_wasb.csv",
218     "n_gem": 9,
219     "play_gem": 51.2,
220     "mean_gem": 5.688888888888889,

```

```

221     "variants": {
222         "baseline": {
223             "n": 1,
224             "play": 174.2742742742743,
225             "mean_dur": 174.2742742742743,
226             "max_dur": 174.2742742742743,
227             "count_ratio": 0.1111111111111111,
228             "matched": 0,
229             "local_only": 0,
230             "gemini_only": 0,
231             "splits": 0,
232             "merges": 1,
233             "mean_iou": 0.0,
234             "gem_rallies_in_merges": 9
235         },
236         "w1": {
237             "n": 2,
238             "play": 173.79045712379047,
239             "mean_dur": 86.89522856189524,
240             "max_dur": 101.88521855188523,
241             "count_ratio": 0.2222222222222222,
242             "matched": 0,
243             "local_only": 0,
244             "gemini_only": 0,
245             "splits": 0,
246             "merges": 2,
247             "mean_iou": 0.0,
248             "gem_rallies_in_merges": 9
249         },
250         "w1+w2": {
251             "n": 2,
252             "play": 173.79045712379047,
253             "mean_dur": 86.89522856189524,
254             "max_dur": 101.88521855188523,
255             "count_ratio": 0.2222222222222222,
256             "matched": 0,
257             "local_only": 0,
258             "gemini_only": 0,
259             "splits": 0,
260             "merges": 2,
261             "mean_iou": 0.0,
262             "gem_rallies_in_merges": 9
263         }
264     },
265     "status": "ok"
266 },
267 {
268     "name": "mahadevpura-2",
269     "vid": "038e0e0915a2",
270     "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-2__038e0e0915a2_wasb.csv",
271     "n_gem": 39,
272     "play_gem": 266.0,
273     "mean_gem": 6.82051282051282,
274     "variants": {
275         "baseline": {
276             "n": 5,
277             "play": 605.9059059059059,
278             "mean_dur": 121.18118118118119,
279             "max_dur": 297.03036369703034,
280             "count_ratio": 0.1282051282051282,
281             "matched": 2,
282             "local_only": 0,
283             "gemini_only": 0,
284             "splits": 0,

```



```

285         "merges": 3,
286         "mean_iou": 0.2945165777074605,
287         "gem_rallies_in_merges": 37
288     },
289     "w1": {
290         "n": 40,
291         "play": 551.0844177510842,
292         "mean_dur": 13.777110443777104,
293         "max_dur": 43.3266599933267,
294         "count_ratio": 1.0256410256410255,
295         "matched": 12,
296         "local_only": 4,
297         "gemini_only": 0,
298         "splits": 4,
299         "merges": 5,
300         "mean_iou": 0.4204774017180335,
301         "gem_rallies_in_merges": 11
302     },
303     "w1+w2": {
304         "n": 40,
305         "play": 551.0844177510842,
306         "mean_dur": 13.777110443777104,
307         "max_dur": 43.3266599933267,
308         "count_ratio": 1.0256410256410255,
309         "matched": 12,
310         "local_only": 4,
311         "gemini_only": 0,
312         "splits": 4,
313         "merges": 5,
314         "mean_iou": 0.4204774017180335,
315         "gem_rallies_in_merges": 11
316     }
317 },
318 "status": "ok"
319 }
320 ]

```

16. assistant (2026-06-16T15:16:13.901Z)

I now have the actual data. Let me verify each numeric claim against the GX010128 W1 record (lines 24-37) and check the cross-video comparison for the "lowest mean window" claim.

GX010128 W1 record:

- `n` = 78, `n\_gem` = 39, `count\_ratio` = 2.0 ?
- `matched` = 35 ?
- `splits` = 4 ?
- `local\_only` = 34 ?
- `mean\_dur` = 7.757... ? 7.8s ?
- `mean\_iou` = 0.529... ? 0.53 ?
- `n\_gem` mean = 5.846s ? 5.85s ?
- `merges` = 0, `gemini\_only` = 0

Now let me check the "lowest of the six" per-video mean window claim. W1 mean_dur per video:

- GX010128: 7.76s
- GX010129: 9.27s
- GX020125: 14.11s
- mahadevpura-0: 10.42s
- mahadevpura-1: 86.90s
- mahadevpura-2: 13.78s

GX010128's 7.76s is indeed the lowest. All cited numbers check out exactly.

Now the adversarial part ? does the data support the *causal mechanism* claimed? The claim asserts the over-segmentation is because "true rallies are short and dense... yet within each rally the shuttle briefly stalls/occludes, so the 12s cap finds a sub-rally inactivity gap and

chops one true rally into two-plus windows."

Let me scrutinize this against the evidence:

1. ```splits` = 4.` The splits metric is the direct, dedicated measure of "one true (Gemini) rally chopped into multiple local windows." It is only `4` out of 39 Gemini rallies and 78 local windows. The claim's stated mechanism (cap+gap splitting *one true rally into two-plus*) is exactly what `splits`` counts, and it fires only 4 times. This is weak support for "over-segments ~2x because... chops one true rally into two-plus."
2. ```local_only` = 34.` The dominant contributor to the 78 windows is 34 local-only windows (windows that match NO Gemini rally at all), plus 35 matched + 4 from splits ? accounts for the count. The 2x over-segmentation is driven overwhelmingly by ```local_only`` spurious windows, not by splitting true rallies. `34 >> 4`. This refutes the claimed primary mechanism.
3. The "12s cap finds a sub-rally inactivity gap and chops" mechanism would manifest as high `splits``. With `splits=4`, the cap-split-inside-a-rally story explains at most ~4 of the ~39 excess windows (78/39). The bulk (34 `local_only``) are windows with no Gemini counterpart ? i.e., the local detector is firing on non-rally activity or fragmenting in a way that doesn't align to Gemini rallies at all.

Let me check the comparison script to confirm exactly what `splits`` and `local_only`` mean, so I'm not misreading the metric semantics.

17. user (2026-06-16T15:16:14.120Z)

```
1      """Local-WASB (no-AI) vs Gemini rally comparison for the Kushagra June-12 set.
2
3      Reads play_segments for the SAME videos from two DBs:
4          - Gemini path  (main DB:  output/sports_indexer.db)          <- produced by --segmenter
gemini
5          - Local path   (compare DB: output/local_compare.db)        <- wasb_hybrid --skip-ai-
handoff
6
7      Both key a video by the SAME video_id (MD5 of the proxy bytes; the local run localizes a
byte
8      copy), so we line them up directly. NEITHER is ground truth ? these videos have no
golden
9      labels ? so this is an AGREEMENT / DIVERGENCE study with Gemini as the strong reference,
NOT a
10     local-accuracy score. Emits a detailed Markdown report.
11
12     Matching: overlap graph -> connected components, each classified by (n_local, n_gemini):
13         (1,1) matched | (1,0) local-only (~false positive) | (0,1) gemini-only (~miss) |
14         (k,1) local-splits-one | (1,k) local-merges-k | (k,m) complex.
15
16     Usage: python scratch/compare_local_vs_gemini.py [--out report.md]
17     """
18     from __future__ import annotations
19
20     import argparse
21     import os
22     import sqlite3
23     import sys
24
25     sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
26     from backend.utils.hashing import compute_video_id # noqa: E402
27
28     _ROOT = r"C:\Users\avidu\Projects\badminton-highlight-indexer"
29     MAIN_DB = os.path.join(_ROOT, "output", "sports_indexer.db")
30     LOCAL_DB = os.path.join(_ROOT, "output", "local_compare.db")
31     PROXY_DIR = r"F:\KhelSutra_work\kushagra_june12_proxies"
32     # The 3 videos compared (must match the Phase-C --video set).
33     VIDEOS = ["mahadevpura-1.MP4", "GX020125.MP4", "mahadevpura-2.MP4"]
34
```

```

35
36 def load_segs(db_path: str, vid: str):
37     if not os.path.exists(db_path):
38         return None
39     con = sqlite3.connect(db_path)
40     try:
41         rows = con.execute(
42             "SELECT start_time, end_time FROM play_segments WHERE video_id=? ORDER BY
start_time",
43             (vid,)).fetchall()
44     except sqlite3.OperationalError:
45         return []
46     finally:
47         con.close()
48     return [(float(s), float(e)) for s, e in rows]
49
50
51 def _fmt_hms(sec: float) -> str:
52     sec = max(0, int(round(sec)))
53     h, r = divmod(sec, 3600)
54     m, s = divmod(r, 60)
55     return f"{h:d}:{m:02d}:{s:02d}" if h else f"{m:d}:{s:02d}"
56
57
58 def overlap(a, b) -> float:
59     return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
60
61
62 def iou(a, b) -> float:
63     inter = overlap(a, b)
64     union = (a[1] - a[0]) + (b[1] - b[0]) - inter
65     return inter / union if union > 0 else 0.0
66
67
68 def components(local, gemini):
69     """Connected components over the bipartite overlap graph (any positive overlap =
edge)."""
70     n, m = len(local), len(gemini)
71     # adjacency
72     ladj = {i: [] for i in range(n)}
73     gadj = {j: [] for j in range(m)}
74     for i, lseg in enumerate(local):
75         for j, gseg in enumerate(gemini):
76             if overlap(lseg, gseg) > 0:
77                 ladj[i].append(j)
78                 gadj[j].append(i)
79     seen_l, seen_g, comps = set(), set(), []
80     for i in range(n):
81         if i in seen_l:
82             continue
83         # BFS over this component
84         lset, gset = set(), set()
85         stack_l, stack_g = [i], []
86         while stack_l or stack_g:
87             while stack_l:
88                 li = stack_l.pop()
89                 if li in lset:
90                     continue
91                 lset.add(li); seen_l.add(li)
92                 for gj in ladj[li]:
93                     if gj not in gset:
94                         stack_g.append(gj)
95             while stack_g:
96                 gj = stack_g.pop()
97                 if gj in gset:

```

```

98             continue
99             gset.add(gj); seen_g.add(gj)
100             for li in gadj[gj]:
101                 if li not in lset:
102                     stack_l.append(li)
103             comps.append((sorted(lset), sorted(gset)))
104 # gemini-only components (no overlapping local)
105 for j in range(m):
106     if j not in seen_g:
107         comps.append([[], [j]])
108         seen_g.add(j)
109 return comps
110
111
112 def analyze(local, gemini):
113     comps = components(local, gemini)
114     buckets = {"matched": [], "local_only": [], "gemini_only": [],
115               "local_splits": [], "local_merges": [], "complex": []}
116     for lset, gset in comps:
117         nl, ng = len(lset), len(gset)
118         if (nl, ng) == (1, 1):
119             buckets["matched"].append((lset, gset))
120         elif (nl, ng) == (1, 0):
121             buckets["local_only"].append((lset, gset))
122         elif (nl, ng) == (0, 1):
123             buckets["gemini_only"].append((lset, gset))
124         elif ng == 1 and nl > 1:
125             buckets["local_splits"].append((lset, gset))
126         elif nl == 1 and ng > 1:
127             buckets["local_merges"].append((lset, gset))
128         else:
129             buckets["complex"].append((lset, gset))
130     return buckets
131
132
133 def report_video(name, vid, gemini, local, out):
134     out.append(f"## {name}")
135     out.append(f"`video_id={vid}`")
136     out.append("")
137     if gemini is None and local is None:
138         out.append("_Neither DB has this video yet._\n")
139         return
140     if gemini is None:
141         out.append("_Gemini (main DB) has no rows for this video yet._")
142         gemini = []
143     if local is None:
144         out.append("_Local (compare DB) has no rows for this video yet._")
145         local = []
146
147     g_tot = sum(e - s for s, e in gemini)
148     l_tot = sum(e - s for s, e in local)
149     out.append(f"| metric | Gemini | Local (WASB no-AI) |")
150     out.append(f"|---|---|---|")
151     out.append(f"| rallies | {len(gemini)} | {len(local)} |")
152     out.append(f"| total play time | {_fmt_hms(g_tot)} | {_fmt_hms(l_tot)} |")
153     gm = (g_tot / len(gemini)) if gemini else 0
154     lm = (l_tot / len(local)) if local else 0
155     out.append(f"| mean rally dur (s) | {gm:.1f} | {lm:.1f} |")
156     out.append("")
157
158     if not gemini or not local:
159         out.append("_ (Need both sides to diff.)_\n")
160         return
161
162     b = analyze(local, gemini)

```

```

163     matched = b["matched"]
164     ious = [iou(local[li[0]], gemini[gj[0]]) for li, gj in matched]
165     mean_iou = sum(ious) / len(ious) if ious else 0.0
166     out.append("**Agreement breakdown** (Gemini = reference, not ground truth):")
167     out.append("")
168     out.append(f"- **matched 1:1**: {len(matched)} (mean IoU {mean_iou:.2f})")
169     out.append(f"- **local-only** (local rally with no Gemini overlap ? likely false
positive, "
170                 f"e.g. warm-up / between-point / held-shuttle): {len(b['local_only'])}")
171     out.append(f"- **gemini-only** (Gemini rally local missed ? likely false negative):
"
172                 f"{len(b['gemini_only'])}")
173     out.append(f"- **local splits one Gemini rally**: {len(b['local_splits'])} groups")
174     out.append(f"- **local merges several Gemini rallies**: {len(b['local_merges'])}
groups")
175     out.append(f"- **complex (many:many)**: {len(b['complex'])} groups")
176     out.append("")
177
178     # Boundary deltas on matched pairs.
179     if matched:
180         sdeltas = [local[li[0]][0] - gemini[gj[0]][0] for li, gj in matched]
181         edeltas = [local[li[0]][1] - gemini[gj[0]][1] for li, gj in matched]
182         out.append(f"Matched-pair boundary deltas (local ? gemini, seconds): "
183                 f"start mean {sum(sdeltas)/len(sdeltas):+.2f} "
184                 f"(min {min(sdeltas):+.1f}, max {max(sdeltas):+.1f}); "
185                 f"end mean {sum(edeltas)/len(edeltas):+.2f} "
186                 f"(min {min(edeltas):+.1f}, max {max(edeltas):+.1f}).")
187         out.append("")
188
189     def _lst(groups, side):
190         items = []
191         for lset, gset in groups:
192             idxs = lset if side == "local" else gset
193             segs = local if side == "local" else gemini
194             for k in idxs:
195                 items.append(f"{{_fmt_hms(segs[k][0])}}?{{_fmt_hms(segs[k][1])}}
({segs[k][1]-segs[k][0]:.0f}s)")
196         return items
197
198     lo = _lst(b["local_only"], "local")
199     go = _lst(b["gemini_only"], "gemini")
200     if lo:
201         out.append(f"<details><summary>local-only windows ({len(lo)})</summary>\n")
202         out.append(", ".join(lo))
203         out.append("\n</details>")
204     if go:
205         out.append(f"<details><summary>gemini-only windows ({len(go)})</summary>\n")
206         out.append(", ".join(go))
207         out.append("\n</details>")
208     out.append("")
209
210
211     # WASB per-frame shuttle visibility observed in the Phase-C run.logs (visible/total
frames).
212     # Evidence that the shuttle SIGNAL is strong ? so divergence is a windowing problem, not
tracking.
213     _VISIBILITY = {"mahadevpura-1.MP4": "98.6% (10305/10448)",
214                   "GX020125.MP4": "72.8% (12992/17848)",
215                   "mahadevpura-2.MP4": "78.8% (29063/36901)"}
216
217
218     def findings(stats):
219         have = [s for s in stats if s["n_gem"] and s["n_loc"]]
220         tg = sum(s["n_gem"] for s in stats)
221         tl = sum(s["n_loc"] for s in stats)

```

```

222     pg = sum(s["play_gem"] for s in stats)
223     pl = sum(s["play_loc"] for s in stats)
224     o = ["--", "", "## Findings ? what the divergence says about the local model", ""]
225     o.append(f"***Headline:** the fully-local WASB path emits FAR FEWER but MUCH LONGER
windows than "
226         f"Gemini ? it does not miss activity, it FAILS TO SEGMENT it. Across
{len(stats)} videos: "
227         f"***Gemini {tg} rallies ({_fmt_hms(pg)} play) vs local {tl} windows
({_fmt_hms(pl)} "
228         f"\\"play\\")**.")
229     o.append("")
230     if have:
231         mg = sum(s["play_gem"] for s in have) / max(1, sum(s["n_gem"] for s in have))
232         ml = sum(s["play_loc"] for s in have) / max(1, sum(s["n_loc"] for s in have))
233         o.append(f"- **Mean window duration: Gemini {mg:.1f}s vs local {ml:.1f}s "
234             f"({ml/mg:.1f}x longer).** Local windows each swallow several rallies +
the dead "
235             f"time between them.")
236         o.append(f"- **Merge groups** (one local window overlapping ?2 Gemini rallies):
"
237             f"{sum(s['merges'] for s in have)}. **Splits** (local fragmenting one
rally): "
238             f"{sum(s['splits'] for s in have)}.")
239         o.append(f"- **Gemini-only (true misses): {sum(s['gemini_only'] for s in
have)}** ? i.e. local "
240             f"rarely misses a region outright; it over-merges instead.")
241         longest = max(stats, key=lambda s: s["max_loc"], default=None)
242         if longest and longest["max_loc"] > 0:
243             o.append(f"- Longest single local window: {longest['max_loc']:.0f}s on
{longest['name']} "
244                 f"(mahadevpura-1's one window = the ENTIRE clip).")
245         o.append("")
246         o.append(f"***Shuttle tracking is NOT the problem.** WASB per-frame visibility was
high "
247             + "; ".join(f"{k.replace('.MP4','')} {v}" for k, v in _VISIBILITY.items())
248             + " ? the trajectory signal is present and dense.")
249         o.append("")
250         o.append(f"***The bottleneck is the velocity windowing**
(`trajectory_to_action_windows`): a frame is "
251             "'active' iff the shuttle is VISIBLE and its per-second-normalized speed
exceeds "
252             "`velocity_threshold=0.02`; active frames within `dead_time_merge_gap=2.0s`
are merged; windows "
253             "shorter than `min_window_duration=2.0s` are dropped. The flaw is
conceptual, not a tuning miss: "
254             "'the shuttle is moving' is treated as 'a rally is in progress.'"
255             "During dead time the shuttle "
256             "stays visible and gently moving (picked up, walked, knock-ups, slow
serves), so those frames "
257             "read as active; the 2 s merge then bridges the brief gaps. Two structural
gaps make it unbounded:")
258         o.append(f"1. **No rally-STATE signal.** The heuristic only has 'is the shuttle
moving' ? nothing that "
259             "says 'is a rally actually happening' (net-crossings, two-sided play,
sustained fast exchange, "
260             "hit sounds). With 96?99% shuttle visibility there are almost no
invisibility breaks to end a "
261             "window, so everything fuses. (NOTE: velocity IS already per-second-
normalized via `fps/dframes`, "
262             "so this is NOT an fps-renormalization bug ? correcting an earlier
hypothesis; fps matters only "
263             "secondarily, via denser sampling keeping more slow-motion frames
'active'.)")
264         o.append(f"2. **No max-window cap / no inactivity split.** Nothing bounds window

```

```

length or forces a "
265         "split on a sustained low-velocity (shuttle-at-rest) stretch, so a single
merge runs to clip length.")
266     o.append("")
267     o.append("## Recommendations (to improve the local model / approach Gemini without
it)")
268     o.append("")
269     o.append("- **Add a rally-STATE signal to gate/split** (the core fix) ? net-
crossings (Gate-A, already "
270         "built), two-sided player motion, and audio hit-rate distinguish 'rally'
from 'shuttle merely "
271         "moving'. Feed them to the learned scorer (no special-casing) so dead-time
windows score low.")
272     o.append("- **Add a `max_window_duration` cap + an inactivity splitter** ? split a
candidate when "
273         "the shuttle velocity stays below threshold for > N s (a rally boundary),
so dead time "
274         "breaks windows instead of bridging them. (A `WindowAnalyzer` per
`ANALYZERS_AND_RECONCILERS.md`.)")
275     o.append("- **Per-venue threshold calibration** against golden labels
(collector?indexer bridge) ? a "
276         "cheap near-term win; tune `velocity_threshold` + `merge_gap` so local
segment counts track the "
277         "golden counts. (velocity is already fps-normalized, so calibrate per-
venue, not per-fps.)")
278     o.append("- **Gate-A (`net_crossings?2`)** to drop windows with no real exchange
(held-shuttle / "
279         "warm-up) ? already built; orthogonal to the over-merge fix.")
280     o.append("- **Durable fix = a learned temporal segmenter** (`rally_seq` / the owned
model) trained "
281         "on golden labels; the velocity heuristic is a placeholder, and this
comparison quantifies "
282         "the gap it must close.")
283     o.append("- **Throughput:** WASB streaming runs ~15 fps on 1080p60, so a 10-min clip
(~37k frames) "
284         "exceeds the 30-min `timeout_sec` (mahadevpura-2 hit it first pass). Raise
the timeout for "
285         "long clips, batch the WASB forward pass, and/or index on 30 fps proxies
(halves frames; "
286         "rally segmentation needs far less than 60 fps).")
287     o.append("")
288     o.append("_Caveat: Gemini is a strong reference, not ground truth; these clips have
no golden labels. "
289         "Treat the numbers as agreement/divergence, not local accuracy. The
directional finding "
290         "(local over-merges; tracking is fine) is robust across all videos
measured._")
291     o.append("")
292     return o
293
294
295     def main():
296         ap = argparse.ArgumentParser()
297         ap.add_argument("--out", default=os.path.join(_ROOT, "output",
"local_vs_gemini_report.md"))
298         args = ap.parse_args()
299
300         out = ["# Local (WASB, no-AI) vs Gemini ? rally comparison",
301             "",
302             "Kushagra/Yash June-12 set (1080p proxies). **Gemini is the reference, NOT
ground "
303             "truth** ? these clips have no golden labels, so this measures
agreement/divergence, "
304             "not local accuracy. Local = `wasb_hybrid --skip-ai-handoff` (WASB shuttle-
trajectory "

```

```

305         "candidate windows emitted directly as rallies; no Gemini).",
306         ""]
307     stats = []
308     for name in VIDEOS:
309         proxy = os.path.join(PROXY_DIR, name)
310         if not os.path.exists(proxy):
311             out.append(f"## {name}\n_proxy missing: {proxy}_\n")
312             continue
313         vid = compute_video_id(proxy)
314         gemini = load_segs(MAIN_DB, vid) or []
315         local = load_segs(LOCAL_DB, vid) or []
316         report_video(name, vid, gemini, local, out)
317         b = analyze(local, gemini) if (gemini and local) else None
318         stats.append({
319             "name": name, "n_gem": len(gemini), "n_loc": len(local),
320             "play_gem": sum(e - s for s, e in gemini), "play_loc": sum(e - s for s, e in
local),
321             "max_loc": max((e - s for s, e in local), default=0.0),
322             "merges": len(b["local_merges"]) if b else 0,
323             "splits": len(b["local_splits"]) if b else 0,
324             "matched": len(b["matched"]) if b else 0,
325             "local_only": len(b["local_only"]) if b else 0,
326             "gemini_only": len(b["gemini_only"]) if b else 0,
327         })
328
329     out += findings(stats)
330     text = "\n".join(out)
331     with open(args.out, "w", encoding="utf-8") as fh:
332         fh.write(text)
333     print(text)
334     print(f"\n[written to {args.out}]")
335
336
337 if __name__ == "__main__":
338     main()
339

```

18. assistant (2026-06-16T15:16:38.488Z)

Now I have the exact metric semantics confirmed (lines 112-130, 169-173):

```

- ``splits`` (``local_splits``): connected components where nl > 1, ng == 1 ? local fragments
ONE Gemini rally into multiple windows. ``This is precisely the mechanism the claim asserts``
(one true rally chopped into two-plus windows).
- ``local_only``: (nl, ng) == (1, 0) ? a local window with NO Gemini overlap ? likely false
positive (warm-up / between-point / held-shuttle).
- ``matched``: 1:1 overlap.

```

This is the decisive distinction. Let me now reason carefully about the claim's logic.

The claim's **``stated causal mechanism``** is: over-segments ~2x **``because``** the 12s cap finds a sub-rally inactivity gap and **``chops one true rally into two-plus windows``**. That mechanism is what **``splits``** counts directly, and **``splits = 4``**.

The 78 local windows decompose (by component bucket) into: 35 matched (1:1) + 34 local_only + 4 split-groups (which contain 78 windows for 4 Gemini rallies) + 0 merges + 0 gemini_only + complex. The arithmetic: 35 + 34 = 69 windows are accounted for as matched and local_only; the remaining 9 windows fall in the 4 split groups (avg ~2.25 windows/group). So the count inflation from 39?78 is dominated by the **``34 local\_only spurious windows``**, with the cap-split mechanism explaining at most ~5 excess windows (9 split-windows covering 4 true rallies = 5 extra).

So the dominant driver of the 2x is **``local\_only false positives (warm-up/between-point/held-shuttle/dead-time activity that the velocity heuristic mistakes for rally)``** ? NOT splitting true rallies via the cap+gap. The claim's headline causal story ("chops one true rally into two-plus windows") is the **``minority``** contributor.

Additionally, the report's own Findings section (lines 250-265) attributes the over-segmentation root cause to "***no rally-STATE signal***" ? the velocity heuristic treating "shuttle is moving" as "rally in progress," firing on dead time. That is the local_only mechanism, not the cap-split mechanism.

Let me also scrutinize the secondary supporting claims:

1. "windows SHORTER than Gemini's 5.85s mean would predict for clean 1:1" ? actually GX010128 W1 mean_dur is **7.76s, which is LONGER than Gemini's 5.85s**, not shorter. The claim's phrasing here is confused/backwards. Local windows are longer on average, yet there are 2x as many of them. If splitting-one-rally-into-pieces were the driver, you'd expect local windows SHORTER than Gemini. They're longer. This actively undercuts the cap-split-inside-rally story and points to local_only windows landing on dead-time/non-rally regions.

2. "per-video mean 7.8s is the lowest of the six" ? TRUE ($7.76 < 9.27 < 10.42 < 13.78 < 14.11 < 86.90$). But "lowest of six" is consistent with *both* mechanisms and doesn't discriminate; it's not evidence for the cap-split story specifically. And note GX020125 (count_ratio 1.64, 0 splits, mean 14.1s) and mahadevpura-2 (4 splits, mean 13.8s) over-segment too without short windows ? weakening the "short windows ? cap splitting" link.

3. "mIoU only 0.53 despite 35 matches" ? mean_iou for the 35 matched is 0.529, true. But mIoU here is computed only over the 35 matched pairs (line 164-165); it reflects boundary imprecision on matched rallies, not the over-segmentation. It is consistent with the claim but doesn't establish the cap-split causation.

****Verdict:**** The cited individual numbers (n=78, count_ratio 2.0, matched 35, splits 4, local_only 34, mean 7.8s lowest, mIoU 0.53, Gemini 5.85s/39) are all accurate. But the claim's central ****causal mechanism is refuted by its own cited evidence****: `splits=4` is small while `local\_only=34` dominates, so the 2x over-segmentation is overwhelmingly driven by spurious windows with no Gemini overlap (dead-time/between-point false positives from the velocity heuristic), not by the 12s cap chopping single true rallies into pieces. The claim also gets the window-length direction backwards (7.8s is longer than Gemini's 5.85s, not "shorter than a clean 1:1 would predict"). The finding is overstated/mis-attributed.

19. user (2026-06-16T15:16:49.713Z)

Structured output provided successfully

20. assistant (2026-06-16T15:16:55.644Z)

I've completed the adversarial verification. The cited file was not at the stated path (`output/local\_vs\_gemini\_6.json` in the worktree) but existed in the parent repo at `C:\Users\avidu\Projects\badminton-highlight-indexer\output\local\_vs\_gemini\_6.json`, which I read along with the metric-defining script `C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\compare\_local\_vs\_gemini.py`.

Verdict: the claim does NOT hold. Every cited scalar is accurate, but the claim's central causal mechanism is refuted by its own data ? `splits=4` is the dedicated count of "one true rally chopped into multiple windows," while `local\_only=34` (spurious windows with no Gemini overlap) dominates the 39?78 inflation. The over-segmentation is driven by dead-time/between-point false positives, not the 12s cap splitting true rallies. The claim also reverses the window-length direction (7.8s is longer than Gemini's 5.85s, not shorter).